MACHINE LEARNING OPERATIONS

Heart Disease Classification Pipeline

End-to-end MLOps implementation and evaluation

Assignment	Course	Author	Date
01	AIMLCZG523	SAI CHANDER NAGAMALLA 2025AE05029	20 October 2026

Executive summary

This project builds a reproducible classification workflow around the processed Cleveland subset of the UCI Heart Disease dataset. It covers data acquisition, exploratory analysis, leakage-safe preprocessing, cross-validated comparison of Logistic Regression and Random Forest, MLflow tracking, automated checks, and a FastAPI inference service with Prometheus metrics.

Across five stratified folds, Logistic Regression achieved the highest mean ROC-AUC at 0.9188. A small tuning search selected C=0.1 and returned a cross-validated ROC-AUC of 0.9208. These are development estimates on a small historical dataset, not evidence of clinical validity or a held-out test score.

The refreshed data analysis, model training and MLflow evidence have been executed locally. Ruff and 12 tests pass. The Docker image built successfully and its isolated API passed readiness, prediction and metrics checks. Kubernetes requires enablement; no remote CI success is claimed before the author pushes.

Project links

Repository: https://github.com/saichandernagamalla/mlops-heart-disease

MLflow: http://127.0.0.1:5000. Verified Docker API and live dashboard: http://localhost:8001/monitor. Kubernetes access: http://localhost:8000 after Docker Desktop Kubernetes is enabled and the LoadBalancer is ready.

1. Problem and dataset

Data origin, target definition, and acquisition choices

The assignment asks for a cloud-ready, monitored API that predicts the presence or absence of heart disease from patient measurements. This implementation uses the processed Cleveland subset rather than merging all four UCI databases. UCI describes 303 Cleveland records and a diagnosis field named num with values 0 through 4.

For binary classification, the target is defined as 0 when num equals zero and 1 when num is greater than zero. Thirteen predictors are retained: age, sex, chest pain type, resting blood pressure, cholesterol, fasting blood sugar flag, resting ECG code, maximum heart rate, exercise-induced angina, ST depression, ST segment slope, major vessels, and thalassemia code.

Data item	Project treatment
Source file	processed.cleveland.data from UCI Heart Disease archive
Rows and fields	303 records; 13 predictors and one binary target
Missing values	Four missing ca entries and two missing thal entries
Target	num=0 maps to 0; num=1,2,3,4 map to 1
Raw storage	data/raw/processed.cleveland.data; excluded from Git
Cleaned data	data/processed/heart.csv; retained with the project

The acquisition script downloads the official archive, extracts only the processed Cleveland file, preserves the raw source locally, converts question marks to missing values, and writes the cleaned CSV. Keeping the transformation in code makes the dataset reproducible from a clean checkout.

Data handling boundary

This small, historical cohort is appropriate for an educational workflow demonstration. It does not represent the current population or current clinical practice. No model output should guide diagnosis, triage, or treatment.

2. Exploratory analysis

Class balance and missingness inform the preprocessing design

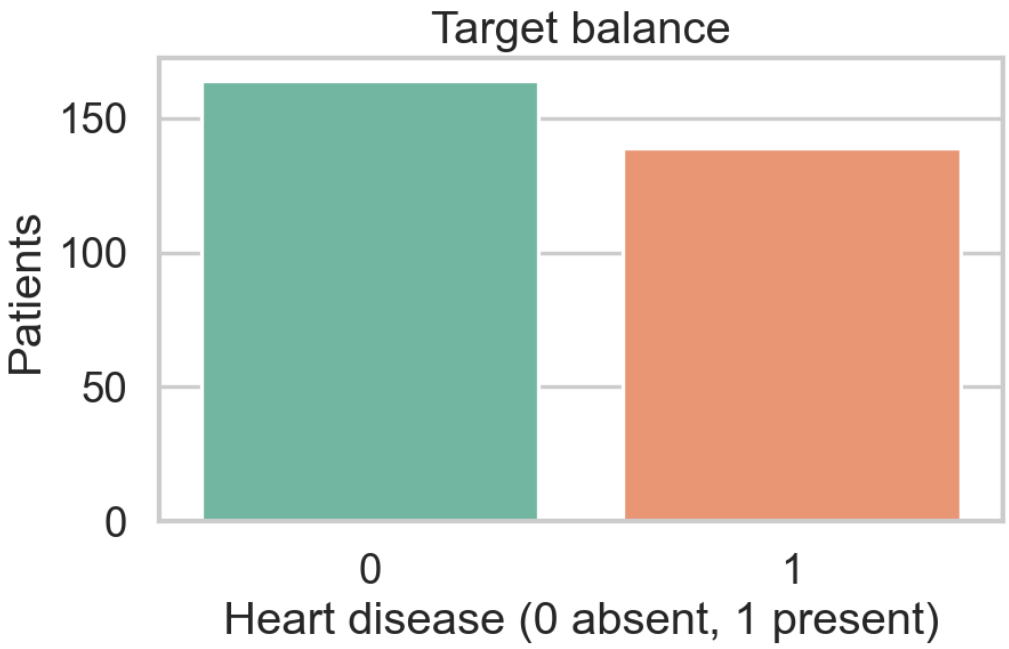


Figure 1. Target class counts for the 303 Cleveland records.

The target distribution is not perfectly balanced. The modeling pipelines therefore use class weighting rather than changing the observed dataset with synthetic oversampling. This keeps the cross-validation folds tied to the source observations and avoids adding generated examples to a small cohort.

Field group	Observed missing cells	Treatment
Numeric predictors	0	Median imputation inside each training fold
Categorical coded predictors	6 total: ca=4, thal=2	Most-frequent imputation inside each training fold
Target	0	Binary mapping from num

Missing values are not filled before splitting. Imputers are part of the sklearn pipeline, so each validation fold receives statistics fitted only on that fold's training partition. This limits leakage and keeps inference behavior consistent with training.

3. Feature distributions and relationships

EDA figures are descriptive and do not imply causation

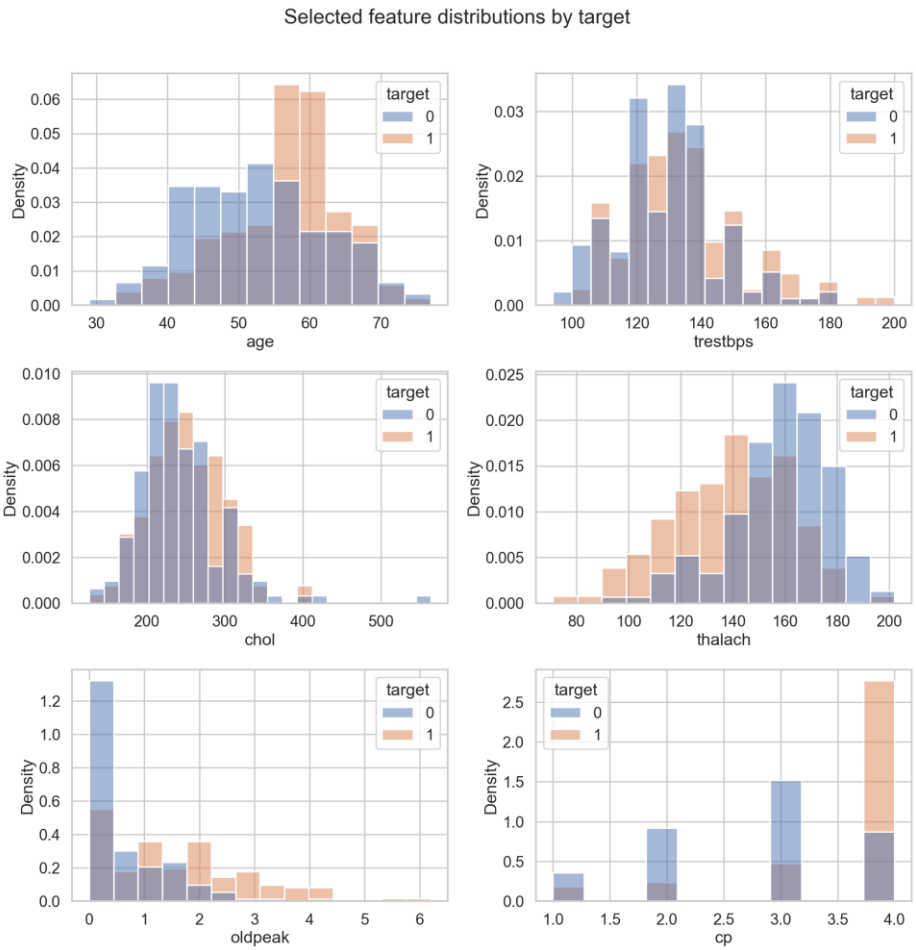


Figure 2. Selected continuous and coded feature distributions, split by target class.

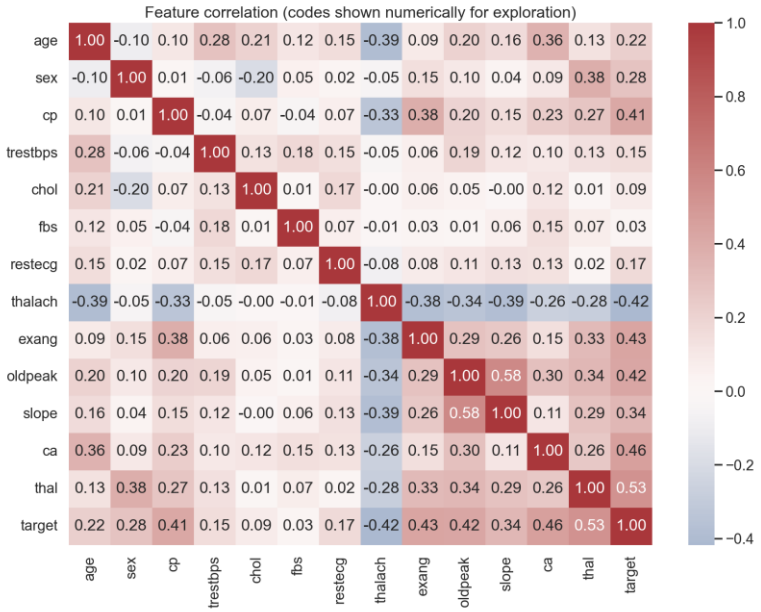


Figure 3. Correlation matrix for selected fields; category codes are displayed numerically for exploration.

The distribution charts support a visual check for spread, overlap, and unusual ranges before modeling. The heatmap is a compact summary of pairwise linear association. For coded categories such as chest pain type, ECG result, slope, and thal, numeric correlation is only exploratory because code values do not necessarily represent equal intervals.

4. Feature engineering and model development

A single preprocessing pipeline is shared across candidates

Five numeric features are median-imputed and standardized. Eight coded categorical features are imputed with their most frequent training-fold value and one-hot encoded. One-hot encoding avoids imposing an artificial ordering on the category codes. Unknown categories at inference are ignored by the encoder rather than causing a request failure.

Stage	Configuration	Reason
Numeric	Median imputation, standard scaling	Robust to a few skewed values; scales for linear model
Categorical	Most-frequent imputation, one-hot encoding	Preserves nominal category meaning
Logistic Regression	class_weight=balanced, max_iter=2000	Interpretable linear baseline with regularization
Random Forest	300 trees, min_samples_leaf=2, balanced weights	Nonlinear tree ensemble comparator

Both estimators are evaluated with the same ColumnTransformer and sklearn Pipeline. The pipeline is fitted within each cross-validation fold, then refitted with the selected parameters on all available rows for packaging. The packaged pipeline contains both preprocessing and classifier steps, so inference accepts the original 13 fields.

Selection approach

The candidate with the higher mean cross-validated ROC-AUC is selected, then a small model-specific grid is searched using the same five-fold scheme. Accuracy, precision, and recall are reported to provide complementary views of classification behavior. No independent test set is held out because the dataset is small; this makes the reported CV results development estimates rather than final generalization proof.

5. Cross-validation results

Five-fold stratified estimates with mean and standard deviation

Candidate	Accuracy	Precision	Recall	ROC-AUC
Logistic Regression	0.8447 ± 0.0554	0.8475 ± 0.0570	0.8050 ± 0.0857	0.9188 ± 0.0213
Random Forest	0.8416 ± 0.0222	0.8447 ± 0.0415	0.8056 ± 0.0438	0.9154 ± 0.0204

Interpretation

Logistic Regression led by mean ROC-AUC (0.9188 compared with 0.9154) and had slightly higher mean accuracy and precision. Random Forest had a marginally higher mean recall and lower fold-to-fold variation in accuracy and recall. The differences are small relative to the observed fold variation and do not establish a meaningful performance advantage.

The selected Logistic Regression candidate was tuned to C=0.1. Its best mean CV ROC-AUC was 0.9208. Because tuning and candidate selection use the same limited dataset, that estimate can be optimistic. A larger independent validation cohort would be required before making any performance claim beyond this educational comparison.

Decision

The model choice follows the stated ROC-AUC criterion, while the similar candidate scores remain visible in the report. This makes the selection rule reproducible and avoids describing the selected model as decisively superior.

6. Experiment tracking and reproducibility

MLflow records the evidence needed to repeat and inspect a run

MLflow experiment uci-heart-disease-assignment contains four refreshed finished runs tagged recording_demo=ready: EDA, Logistic Regression, Random Forest and final_tuned_model. Candidate runs log estimator parameters, seed, fold metrics, mean and standard deviation, out-of-fold confusion matrices and ROC curves.

Artifact	Location or contents
Reusable fitted pipeline	artifacts/model.joblib
Training summary	artifacts/training_summary.json
MLflow local store	mlruns/ (ignored by Git; inspect with mlflow ui)
EDA outputs	reports/figures/ and descriptive statistics CSV
Randomness	Seed 42 in stratified folds and estimators

```
Architecture: UCI -> acquisition -> CSV -> EDA + MLflow CV -> fitted pipeline
GitHub Actions -> lint / tests / EDA / training -> run artifacts
Docker -> Kubernetes LoadBalancer -> FastAPI -> metrics / dashboard / logs
```

The final run a7f870e2af164e1ea71d50eff5ee422f logs the reusable model, inferred signature, input example, training summary, tuning results and diagnostic plots. Its tuned CV ROC-AUC is 0.9208 with C=0.1; this remains a development estimate, not independent validation.

Reproduction commands

Create a Python 3.11 virtual environment, install requirements.txt, then run python -m scripts.fetch_data, python -m scripts.eda, and python -m scripts.train from the repository root. Training regenerates both the pipeline artifact and JSON summary. Use python -m scripts.train --quick only for the reduced CI smoke run.

7. Automated checks and CI

Local verification: Ruff passes; 12 tests pass. The formerly failing pytest entry point now succeeds after adding the project-root import configuration. Remote CI still requires the author's commit and push.

Check	Result	Scope
Ruff lint	Passed	ruff check src scripts tests
Unit tests	12 passed	Data, both model pipelines, serialization, HTTP API, metrics and request logs
EDA run	Completed	Three figures plus descriptive and missing-value tables
Training run	Completed	Both candidates compared; model, summary, and MLflow artifacts saved
GitHub Actions	Pending push	Import fix verified locally; new remote run requires push

The latest observed remote run 39466244558 failed at pytest collection. Reproduction showed ModuleNotFoundError: No module named 'src'. pytest.ini now sets the project import root, and CI uses python -m pytest. The workflow runs lint, tests, acquisition, EDA and training, then uploads model, MLflow, figure and JUnit artifacts. The historical screenshot below shows failed runs, not a passing result.

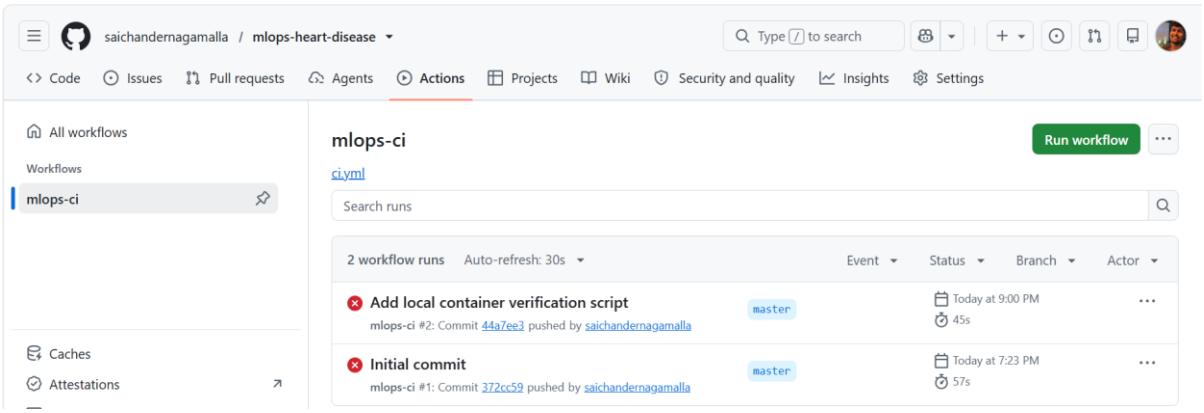


Figure 4. Historical GitHub Actions failures; a successful rerun must be captured after pushing the verified local fix.

Test coverage

- Data loader test checks the schema, conversion of a nonzero diagnosis to the positive class, and retention of a question-mark field as missing.
- API function tests check health status without a loaded model and the prediction, label, and positive-class probability response shape.
- The current Docker build and isolated API checks passed. The running heart-api-demo container serves localhost:8001. Its sample prediction is 0 (negative), with positive-class confidence 0.33645. Model readiness and Prometheus metrics returned HTTP 200.

8. API, packaging, and monitoring

Service contract and current execution evidence

Route	Purpose	Observed output
GET /health	Model availability indicator	HTTP 200; model_loaded=true in the recording container
POST /predict	Validated patient fields to class and probability	Class 0; negative; confidence_positive=0.33645
GET /metrics	Prometheus request counters and latency histogram	HTTP 200; success count 1; latency count 1

POST /predict accepts 13 measurements and returns a binary prediction, label and positive-class confidence. The schema enforces original UCI codes: cp=1..4, slope=1..3, ca=0..3 and thal in {3,6,7}. GET /health returns 503 without a model. The confidence is uncalibrated and must not guide clinical decisions.

Container packaging

The image built and ran successfully with pinned dependencies and non-root UID 10001. The verification script checked readiness, prediction and metrics. A trusted HTTPS index resolved PyPI TLS failures without disabling certificates. The recording container publishes host-only port 8001; Kubernetes uses a LoadBalancer on port 8000 after cluster enablement.

Monitoring

GET /monitor is a live dashboard refreshing every two seconds. The fresh container recorded one successful prediction, latency count 1 and latency sum 0.016678784 seconds. JSON request logs record route, method, status and latency, never patient feature values. Production operation still needs durable collection, authentication, alerts, drift checks and external validation.

9. Deployment, limitations, and completion evidence

Verified status and remaining submission actions

Assignment evidence	Status
Data, EDA, candidate models, CV, MLflow	Executed locally; artifacts and figures generated
Preprocessing package and API implementation	Implemented; unit-tested
Lint and unit tests	Ruff clean; 12 tests passed
Docker image and local container proof	Verified Docker build; health, prediction and metrics passed
Kubernetes/cloud deployment	Pending Docker Desktop Kubernetes enablement
Screenshots and short pipeline video	Capture actual screenshots and record the narrated video

Limitations and next steps

The dataset is small and historical, missingness is concentrated in two fields, and its population and measurements may not reflect current practice. Fold estimates vary, feature correlations do not imply causal effects, and the selected model has not been independently validated or calibrated. The API is an educational demo and must not be used for clinical care.

- Docker build and endpoints are verified. Capture remaining genuine screenshots; enable Docker Desktop Kubernetes and verify its LoadBalancer before claiming cluster deployment.
- Deploy the image to Minikube or an approved cloud environment; record the endpoint and deployment screenshot.
- Push the local fixes to obtain a genuine passing Actions run; record the narrated walkthrough using docs/VIDEO_CHECKLIST.md and add the real video link.
- Review the analysis in your own words and complete only the remaining verified evidence and video link.

References

UCI Machine Learning Repository. Heart Disease dataset. DOI: 10.24432/C52P4X. https://doi.org/10.24432/C52P4X

Machine Learning Operations (MLOps) AIMLCZG523, Assignment 01 instructions supplied for this project.

GitHub repository: https://github.com/saichandernagamalla/mlops-heart-disease